Estruturas de Dados — Entrega 2

1. Introdução e escopo

Este documento identifica, no código da Entrega 2 do projeto Collision, onde aparecem vetores, listas, percorrimento de listas, acesso por chave (tabela hash) e a notação Big-O associada a cada trecho. Para cada trecho são apresentados: a localização, o código comentado, a função no sistema, a análise de complexidade e a boa prática de estrutura de dados que ele ilustra.

Escopo analisado. Todos os arquivos de código do projeto (JavaScript, JSX e HTML), excluindo `node_modules`, imagens e CSS. Somente o **Frontend** (React + Vite) contém código.

Convenções de notação: **n** = quantidade de itens de uma lista; **m** = caracteres de um texto; **k** = quantidade de chaves de um objeto; **L** = tamanho do texto digitado em um formulário; **q** = tamanho da query string da URL.

2. Mapa do projeto

Marcadores: 📋 lista ou vetor declarado · 🔁 percorrimento de lista · 🔑 acesso por chave (objeto usado como dicionário).

```
Entrega 2/
├── Backend/
│
└── Frontend/
    ├── index.html · login.html · dashboard.html
    ├── package.json · vite.config.js              (configuração)
    └── src/
        ├── main.jsx · App.jsx · Rotas.jsx
        ├── data/                                  ★ LISTAS E DICIONARIOS DE DADOS
        │   ├── dadosAdmin.js        📋 revisoes, bimestres, atalhos
        │   ├── dadosProfessor.js    📋 turmas, recentes
        │   ├── dadosResponsavel.js  📋 recentes
        │   ├── dadosDashboard.js    📋🔑 objeto por perfil com menu[] e indicadores[]
        │   └── usuariosMock.js      🔑 dicionário perfil → usuário
        ├── componentes/
        │   ├── Cabecalho.jsx        📋🔁 secoes[] + for...of
        │   └── dashboard/
        │       ├── ConteudoAdmin.jsx        🔁 3 × .map
        │       ├── ConteudoProfessor.jsx    🔁 2 × .map
        │       ├── ConteudoResponsavel.jsx  🔁 1 × .map
        │       ├── MenuLateral.jsx          🔁 .map
        │       ├── MenuPerfil.jsx           📋🔁 perfis[] + .map
        │       └── CartaoIndicador.jsx      🔑 leitura por posição
        ├── paginas/
        │   ├── Dashboard.jsx        🔁🔑 .map + dicionário conteudos
        │   └── login.jsx            🔁🔑 Object.keys + chave dinâmica [name]
        ├── servicos/servicoUsuario.js   🔑 busca por perfil
        └── utilitarios/nomes.js         🔁 split → slice → map → join
```

Os vetores e listas de dados ficam concentrados em `src/data/`. As demais pastas consomem esses dados.

3. Boas práticas de estrutura de dados usadas como critério

1. **Separar dados de interface.** Os dados ficam em `src/data/` e os componentes apenas os consomem. Trocar os dados de teste por uma API não exige alterar as telas.
2. **Escolher a estrutura pelo padrão de acesso.** Sequência com ordem e repetição pede um vetor; busca por identificador pede um dicionário, com acesso O(1) em média.
3. **Indexar em vez de percorrer.** Escolher por chave (`conteudos[perfil]`) evita cadeias de `if/else` e buscas O(n).
4. **Documentar registros posicionais.** Vetores de vetores só são legíveis se o formato de cada posição estiver descrito (JSDoc). A alternativa, com objetos de campos nomeados, é discutida na seção 9.
5. **Comparar o custo com o tamanho real dos dados.** Com n pequeno o custo prático é irrelevante, mas a análise mostra como o código reagiria se n crescesse.
6. **Estado imutável tem custo.** Copiar um objeto com spread custa O(k), aceitável quando k é pequeno.
7. **Chaves estáveis em listas renderizadas.** Uma `key` única permite ao React atualizar apenas o que mudou.

4. Vetores e listas (pasta `src/data/`)

4.1 `src/data/dadosAdmin.js`

```
/**
 * Fila de revisão exibida no painel do administrador.
 * Estrutura: vetor de vetores (lista de registros com posições fixas).
 * Formato de cada registro:
 * [iniciais, nome, turma, detalhe, data, estado, cor]
 */
export const revisoes = [
  ["ML", "Monkey D. Luffy", "Matematica . 8º A", "enviado por Prof. Chico", "Hoje, 10:32", "Pendente", "laranja"],
  ["RZ", "Roronoa Zoro", "Geografia . 8º A", "Enviado por Prof. AArlong", "Ontem, 14:20", "Pendente", "laranja"],
```

```
  ["LK", "Lucas Kazuo", "Portugues . 9º A", "Devolvido Para ajustes", "18 set 09:12", "Devolvido", "azul"],
];

/**
 * Calendário dos bimestres, em ordem cronológica.
 * Formato: [nome, periodo, estado]
 * Estados possíveis: "Concluído" | "Aberto" | "Em breve"
 */
export const bimestres = [
  ["1º bimestre", "01 fev – 30 abr", "Concluído"],
  ["2º bimestre", "01 mai – 31 jul", "Concluído"],
  ["3º bimestre", "01 ago – 30 set", "Aberto"],
  ["4º bimestre", "01 out – 20 dez", "Em breve"],
];

/**
 * Atalhos da área "Acesso rápido".
 * Formato: [titulo, descricao, secaoDeDestino, nomeDoIcone]
 */
export const atalhos = [
  ["Cadastros e usuários", "Gerencie usuários, alunos, professores e responsáveis.", "Usuários", "imgSidebarUsuarios"],
  ["Vínculos acadêmicos", "Configure turmas, disciplinas e vínculos professor-turma.", "Turmas e vínculos", "imgIconeTurm
asAtivas"],
  ["Relatórios e exportação", "Consulte relatórios e exporte dados em Excel.", "Relatórios", "imgIconeRelatoriosEExportac
ao"],
];
```

Função no sistema: guarda os dados exibidos no painel do administrador: fila de revisão, linha do tempo dos bimestres e botões de acesso rápido. Hoje são dados fixos que simulam o que viria de um banco de dados.

Big-O: criar cada vetor literal custa O(n), uma única vez, quando o módulo é carregado. Ler uma posição (`revisoes[0]`, `registro[1]`) é **O(1)**, pois o acesso é direto pelo índice. Percorrer o vetor inteiro é O(n) (seção 5). Espaço: O(n).

Boa prática: os dados estão isolados da interface, e o formato de cada registro está documentado no comentário, o que reduz o risco de confundir posições.

4.2 `src/data/dadosProfessor.js` e `src/data/dadosResponsavel.js`

```
// dadosProfessor.js
/** Turmas do professor. Formato: [nomeDaTurma, disciplina, quantidadeDeAlunos] */
export const turmas = [
  ["8º A", "Matemática", "32 alunos"],
  ["9º B", "Portugues", "29 alunos"],
];

/** Acompanhamentos recentes. Formato: [iniciais, nomeDoAluno, turma, data, estado, cor] */
export const recentes = [
  ["RZ", "Roronoa Zoro", "8º A", "Hoje 10:30", "Feedback Enviado", "verde"],
  ["NU", "Naruto Uzumaki", "8º B", "Ontem 14:23", "Em Rascunho", "laranja"],
  ["LY", "Light Yagami", "9º C", "Ontem, 16:32", "Retorno Recebido", "azul"],
];

// dadosResponsavel.js
/** Acompanhamentos do responsável. Formato: [titulo, data, estado, cor, nomeDoIcone] */
export const recentes = [
  ["Feedback De matematica", "Hoje, 10:30", "Novo!", "laranja", "imgIconeChatOrange"],
  ["Projeto de Ciências", "Ontem 14:32", "Visto", "azul", "imgIconeFlaskGreen"],
  ["Atividade Historia", "18 set, 12:23", "Respondido", "verde", "imgIconeDocumentGreen"],
];
```

Função no sistema: alimentam os painéis “Minhas turmas” e “Acompanhamentos recentes” do professor e o painel “Acompanhamentos Recentes” do responsável.

Big-O: mesma análise do item 4.1: leitura por índice O(1), criação O(n) uma única vez, espaço O(n).

Boa prática: cada perfil tem seu próprio arquivo de dados, o que mantém cada lista pequena e fácil de manter.

4.3 `src/data/dadosDashboard.js`

```
/**
 * Dicionário de configuração do dashboard, indexado pelo perfil do usuário.
 * Chaves: "professor" | "responsavel" | "admin"
 * Cada perfil tem:
 *  - textos de apresentação (descricao, botao, destinoBotao)
 *  - menu: vetor de [tituloDoItem, nomeDoIcone]
 *  - indicadores: vetor de [titulo, valor, legenda, nomeDoIcone]
 */
export const dadosDashboard = {
  professor: {
    descricao: "Acompanhe turmas e o desenvolvimento de alunos.",
    botao: "Novo acompanhamento",
```

```
    destinoBotao: "acompanhamento",
    menu: [
      ["Início", "imgSidebarInicio"],
      ["Minhas Turmas", "imgSidebarMinhasTurmas"],
      ["Alunos", "imgSidebarAlunos"],
      ["Acompanhamentos", "imgSidebarAcompanhamentos"],
      ["Atividades", "imgSidebarAtividades"],
      ["Retornos", "imgSidebarRetornos"],
    ],
    indicadores: [
      ["Minhas turmas", "4", "Turmas ativas neste ano", "imgIconeTeamBlue"],
      ["Alunos", "126", "alunos sob sua responsabilidade", "imgIconeUserBlue"],
      ["Acompanhamentos pendnetes", "3", "aguardandno envio", "imgIconeDocumentOrange"],
    ],
  },
  // responsavel: mesma estrutura (menu com 7 itens, indicadores com 3)
  // admin:       mesma estrutura (menu com 10 itens, indicadores com 3)
};
```

Função no sistema: define, para cada perfil, quais itens aparecem no menu lateral e quais cartões de números aparecem no topo do dashboard.

Big-O: buscar um perfil (`dadosDashboard["professor"]`) é **O(1)** em média; ler `menu[i]` ou `indicadores[i]` é O(1). Espaço: O(p · (a + b)), com p = perfis (3), a = itens do menu e b = indicadores.

Boa prática: combina dicionário (busca por perfil) com vetores (sequência ordenada de itens), cada estrutura no papel para o qual é adequada.

4.4 `MenuPerfil.jsx` (linhas 3 a 7) e `Cabecalho.jsx` (linha 23)

```
// MenuPerfil.jsx — perfis do modo demonstração. Formato: [chaveDoPerfil, rotuloExibido]
const perfis = [
  ["admin", "Administrador"],
  ["professor", "Professor"],
  ["responsavel", "Responsável"],
];

// Cabecalho.jsx — ids das seções da Home acompanhadas na rolagem.
// A ordem importa: segue a ordem em que as seções aparecem na página.
const secoes = ["inicio", "recursos", "noticias"];
```

Função no sistema: `perfis` define as opções do menu de troca de perfil de demonstração; `secoes` define quais seções da página inicial o cabeçalho acompanha para destacar o link ativo.

Big-O: acesso por índice O(1). Ambas têm tamanho fixo de 3 itens.

Boa prática: constantes declaradas fora das funções não são recriadas a cada renderização, e a ordem do vetor está documentada.

5. Percorrimento de listas

5.1 `ConteudoAdmin.jsx` — três `.map` (linhas 14, 51 e 69)

Trecho resumido; a lógica de percorrimento é a original.

```
{/* Linha 14 — percorre a fila de revisão e cria uma linha clicável por registro */}
{revisoes.map(([iniciais, nome, turma, detalhe, data, estado, cor]) => (
  <button className="dash-linha" type="button"
          key={nome} /* identifica o item para o React atualizar só o que mudou */
          onClick={() => onNavegar("Revisão")}>
    <span className="dash-iniciais">{iniciais}</span>
    {/* ... nome, turma e detalhe ... */}
    <small className="dash-data">{data}</small>
    {/* A cor escolhe a classe CSS da etiqueta de estado */}
    <span className={`dash-status dash-status--${cor}`}>{estado}</span>
  </button>
))}

{/* Linha 51 — percorre os bimestres e monta a linha do tempo */}
{bimestres.map(([nome, datas, estado]) => (
  <div className={`dash-bimestre ${estado === "Aberto" ? "dash-bimestre-atual" : ""}`} key={nome}>
    {/* "✓" apenas nos bimestres concluídos */}
    <span className="dash-ponto">{estado === "Concluído" ? "✓" : ""}</span>
    <strong>{nome}</strong>
    <small>{datas}</small>
    <small>{estado}</small>
  </div>
))}

{/* Linha 69 — percorre os atalhos e cria um botão por atalho */}
{atalhos.map(([titulo, texto, destino, icone]) => (
  <button className="dash-linha" type="button" key={titulo}
          onClick={() => onNavegar(destino)}> {/* leva à seção indicada no atalho */}
    <img src={`/img/dashboard/admin/${icone}.png`} alt="" />
```

```
    <strong>{titulo}</strong>
    <small>{texto}</small>
  </button>
))}
```

Função no sistema: transforma cada lista de dados em elementos visuais da tela do administrador: para cada item do vetor, o `.map` devolve um botão ou bloco já preenchido.

Big-O: cada `.map` é **O(n)**, pois visita os n itens uma vez. O trabalho dentro de cada volta é constante (desestruturar de 3 a 7 posições fixas e montar um elemento, O(1) por item). Os três `.map` rodam em sequência, sem aninhamento; o custo total é $O(n_1 + n_2 + n_3)$, que continua sendo O(n). Hoje: $n_1 = 3$, $n_2 = 4$, $n_3 = 3$.

Boa prática: listas percorridas lado a lado, sem laços aninhados, evitam crescimento quadrático; a `key` única mantém a atualização da tela eficiente.

5.2 `ConteudoProfessor.jsx` (linhas 14 e 41) e `ConteudoResponsavel.jsx` (linha 33)

```
{/* ConteudoProfessor, linha 14 — percorre as turmas do professor */}
{turmas.map(([nome, materia, alunos]) => (
  <button className="dash-linha dash-linha-turma" type="button" key={nome}
          onClick={() => onNavegar("Minhas Turmas")}>
    <strong>{nome}</strong>
    <small>{materia}</small>
    <small>{alunos}</small>
    <span className="dash-linha-link">Ver turma →</span>
  </button>
))}

{/* ConteudoProfessor, linha 41 — percorre os acompanhamentos recentes */}
{recentes.map(([iniciais, nome, turma, data, estado, cor]) => (
  <button className="dash-linha" type="button" key={nome}
          onClick={() => onNavegar("Acompanhamentos")}>
    <span className="dash-iniciais">{iniciais}</span>
    <strong>{nome}</strong>
    <small className="dash-data">{data}</small>
    <span className={`dash-status dash-status--${cor}`}>{estado}</span>
  </button>
))}

{/* ConteudoResponsavel, linha 33 — percorre os acompanhamentos do responsável */}
{recentes.map(([titulo, data, estado, cor, icone]) => (
  <button className="dash-linha" type="button" key={titulo}
          onClick={() => onNavegar("Acompanhamentos")}>
    {/* O nome do ícone vem do próprio registro */}
    <img src={`/img/dashboard/responsavel/${icone}.png`} alt="" />
    <strong>{titulo}</strong>
    <small>{data}</small>
    <span className={`dash-status dash-status--${cor}`}>{estado}</span>
  </button>
))}
```

Função no sistema: mostram, nas telas do professor e do responsável, as listas de turmas e de acompanhamentos, com um botão clicável por item.

Big-O: cada `.map` é **O(n)**. No professor, $O(n_1 + n_2) = O(n)$, com $n_1 = 2$ e $n_2 = 3$; no responsável, $n = 3$.

Boa prática: a desestruturação nos parâmetros (`([nome, materia, alunos]) =>`) dá nome a cada posição e deixa o código autoexplicativo.

5.3 `MenuLateral.jsx` (linha 7) e `MenuPerfil.jsx` (linha 29)

```
{/* MenuLateral — percorre os itens do menu e cria um botão por item */}
{itens.map(([titulo, icone]) => (
  <button key={titulo} type="button"
    /* Marca como ativo o item cujo título é igual à seção atual */
    className={`dash-item-menu ${secao === titulo ? "dash-item-ativo" : ""}`}
    /* Acessibilidade: informa a leitores de tela qual é a página atual */
    aria-current={secao === titulo ? "page" : undefined}
    onClick={() => onSelecionar(titulo)}>
    <img src={`/img/dashboard/${perfil}/${icone}.png`} alt="" />
    <span>{titulo}</span>
  </button>
))}

{/* MenuPerfil — percorre os perfis e cria um botão por perfil */}
{perfis.map(([perfil, nome]) => (
  <button key={perfil} type="button"
    disabled={usuario.perfil === perfil} /* desabilita o perfil já em uso */
    onClick={() => {
      onTrocarPerfil(perfil); // avisa o dashboard para trocar de perfil
      setPerfilAberto(false); // fecha o popover
    }}>
    {nome}
```

```
  </button>
))}
```

Função no sistema: `MenuLateral` desenha o menu lateral, que muda conforme o perfil (6, 7 ou 10 itens, vindos de `dadosDashboard[perfil].menu`). `MenuPerfil` monta o menu suspenso de troca de perfil.

Big-O: ambos **O(n)**. A comparação `secao === titulo` em cada item é O(1), então o total permanece O(n), com n entre 6 e 10 no menu lateral e n = 3 nos perfis.

Boa prática: o menu é gerado a partir de dados, e não escrito item a item; adicionar um item exige alterar apenas o vetor.

5.4 `Dashboard.jsx` — dicionário `conteudos` e `.map` de indicadores

```
/**
 * Dicionário que associa cada perfil ao componente de conteúdo correspondente.
 * Evita uma cadeia de if/else: a escolha é feita direto pela chave.
 */
const conteudos = {
  professor: ConteudoProfessor,
  responsavel: ConteudoResponsavel,
  admin: ConteudoAdmin,
};

// ...dentro do componente: busca o componente certo pela chave do perfil
const Conteudo = conteudos[usuario.perfil];

// ...no retorno: percorre os indicadores do perfil e cria um cartão por indicador
{dados.indicadores.map((indicador) => (
  <CartaoIndicador
    key={indicador[0]} /* o título do indicador serve de identificador único */
    perfil={usuario.perfil}
    indicador={indicador}
  />
))}
```

Função no sistema: o dicionário decide qual painel desenhar (professor, responsável ou admin); o `.map` desenha os três cartões de números do topo.

Big-O: `conteudos[usuario.perfil]` é **O(1)** em média; `indicadores.map` é **O(n)**, com n = 3.

Boa prática: um dicionário no lugar de `if/else` mantém a escolha em tempo constante e torna a inclusão de um novo perfil uma alteração de uma linha.

5.5 `Cabecalho.jsx` — `for...of` (linhas 23 a 38)

```
// Ids das seções acompanhadas, na ordem em que aparecem na página.
const secoes = ["inicio", "recursos", "noticias"];

// Descobre qual seção está visível e atualiza o link destacado do menu.
function atualizarSecao() {
  // Margem de 150px: a seção vira ativa um pouco antes de chegar ao topo.
  const limite = window.scrollY + 150;
  let atual = "inicio"; // valor padrão: o topo da página

  // Percorre cada seção, da primeira para a última.
  for (const id of secoes) {
    const elemento = document.getElementById(id);

    // Se a seção já passou do ponto de rolagem, ela é a candidata a "ativa".
    // Como a lista segue a ordem da página, a última que passar no teste vence.
    if (elemento && elemento.offsetTop <= limite) {
      atual = id;
    }
  }

  setSecaoAtiva(atual);
}
```

Função no sistema: a cada rolagem ou redimensionamento, descobre em qual seção a pessoa está e destaca o link correspondente no menu (“Início”, “Recursos” ou “Notícias”).

Big-O: o laço é **O(n)**, com n = 3. Dentro de cada volta, `getElementById` e a comparação são tratados como O(1); na prática, `offsetTop` obriga o navegador a calcular o layout da página. Como a função roda a cada evento de rolagem, manter a lista pequena e fixa mantém o custo praticamente constante.

Boa prática: funções ligadas a eventos frequentes devem percorrer listas curtas. Aqui a lista é fixa em 3 itens.

5.6 `src/utilitarios/nomes.js`

```
/**
 * Gera as iniciais (até 2 letras) de um nome.
 * Exemplo: "lucas Kazuo" -> "LK"
 *
 * @param {string} nome - nome completo da pessoa.
 * @returns {string} iniciais em maiúsculas.
 */
```

```
export function obterIniciais(nome) {
  return nome
    .trim()                          // remove espaços nas pontas
    .split(/\s+/)                    // quebra em palavras (um ou mais espaços)
    .slice(0, 2)                     // mantém só as duas primeiras palavras
    .map((parte) => parte.charAt(0))  // pega a primeira letra de cada palavra
    .join("")                        // junta as letras em um só texto
    .toUpperCase();                  // padroniza em maiúsculas
}

/**
 * Retorna a primeira palavra do nome.
 * Exemplo: "Henrique Silva" -> "Henrique"
 */
export function obterPrimeiroNome(nome) {
  return nome.trim().split(/\s+/)[0];
}
```

Função no sistema: gera as iniciais do avatar do menu de perfil (`MenuPerfil.jsx`) e o primeiro nome usado na saudação (`Saudacao.jsx`).

Big-O: `trim` e `split` percorrem o texto inteiro: **O(m)**. Depois do `slice(0, 2)`, restam no máximo 2 palavras, e `map`, `join` e `toUpperCase` trabalham sobre 2 itens (O(1)). Total de `obterIniciais`: **O(m)**; `obterPrimeiroNome` também é **O(m)**.

Boa prática: funções puras e pequenas (mesma entrada, mesma saída, sem efeito colateral) em uma pasta de utilitários são fáceis de testar e reutilizar. O `slice` antecipado reduz o trabalho das etapas seguintes.

5.7 `login.jsx` — `Object.keys` (linhas 27 a 49)

```
/**
 * Valida o formulário de login e, se estiver tudo certo, vai para o dashboard.
 * @param {Event} event - evento de envio do formulário.
 */
function enviarFormulario(event) {
  event.preventDefault(); // impede o recarregamento da página

  // Objeto que acumula as mensagens de erro: { campo: "mensagem" }.
  const novosErros = {};

  // Regra 1: o login não pode estar vazio.
  if (!campos.login.trim()) {
    novosErros.login = "Digite seu login.";
  }

  // Regra 2: a senha é obrigatória e precisa ter ao menos 6 caracteres.
  if (!campos.senha.trim()) {
    novosErros.senha = "Digite sua senha.";
  } else if (campos.senha.trim().length < 6) {
    novosErros.senha = "A senha deve ter pelo menos 6 caracteres.";
  }

  setErros(novosErros);

  // Se existir qualquer chave no objeto de erros, interrompe o envio.
  if (Object.keys(novosErros).length > 0) {
    return;
  }

  navegar("/dashboard.html");
}
```

Função no sistema: impede o acesso ao dashboard enquanto login e senha não passarem na validação, e exibe a mensagem de erro de cada campo.

Big-O: `Object.keys(novosErros)` é **O(k)**, com k ≤ 2 (praticamente constante). `trim()` é O(L). Total da função: **O(L)**, dominado pelo `trim` do login e da senha.

Boa prática: acumular os erros em um objeto indexado pelo nome do campo permite validar tudo de uma vez e exibir cada mensagem junto ao campo certo (validação antecipada, antes de navegar).

6. Acesso por chave e tabela hash (implícita)

Objetos JavaScript guardam pares chave → valor e permitem buscar pela chave, que é a ideia central de uma tabela hash. O motor do navegador faz isso internamente; o projeto **não implementa** uma tabela hash própria (função de hash, tratamento de colisões, `put` e `get`).

6.1 `usuariosMock.js` e `servicoUsuario.js`

```
// usuariosMock.js — usuários de demonstração indexados pelo perfil (perfil -> dados do usuário)
export const usuariosMock = {
  admin:       { id: 1, nome: "lucas Kazuo", perfil: "admin" },
  professor:   { id: 2, nome: "chico",       perfil: "professor" },
  responsavel: { id: 3, nome: "Henrique",    perfil: "responsavel" },
};
```

```
// servicoUsuario.js
// Pausa a execução por "ms" milissegundos (simula a espera de um servidor).
function esperar(ms) {
  return new Promise((resolver) => setTimeout(resolver, ms));
}

/**
 * Simula a busca dos dados do dashboard em um servidor.
 * @param {string} perfil - "admin" | "professor" | "responsavel".
 * @returns {Promise<{usuario: object, dados: object}>}
 * @throws {Error} se o perfil não existir ou se a falha simulada estiver ativa.
 */
export async function carregarDashboardDemo(perfil) {
  await esperar(700); // simula o tempo de resposta

  // Use ?simularErro=1 no endereço para demonstrar o estado de erro.
  const simularErro = new URLSearchParams(window.location.search).get("simularErro");
  if (simularErro === "1") {
    throw new Error("Falha simulada ao buscar os dados. Tente novamente.");
  }

  // Duas buscas por chave: uma no dicionário de usuários, outra no de dados da tela.
  const usuario = usuariosMock[perfil];
  const dados = dadosDashboard[perfil];

  // Se qualquer uma não existir, o perfil é inválido.
  if (!usuario || !dados) {
    throw new Error(`Não encontramos dados para o perfil "${perfil}".`);
  }

  return { usuario, dados };
}
```

Função no sistema: é a camada de dados do Frontend. Entrega ao dashboard o usuário e os dados do perfil escolhido. Quando o Backend existir, a chamada real à API entra no lugar dos dados de teste.

Big-O: as duas buscas por chave são **O(1)** em média e não dependem de quantos usuários existem; em um vetor, achar o mesmo usuário exigiria percorrer a lista (O(n)). `URLSearchParams(...).get(...)` percorre a query string: O(q), normalmente pequena. O `esperar(700)` é um atraso de tempo, não complexidade algorítmica. Total: **O(q)**, praticamente constante.

Boa prática: isolar o acesso a dados em uma camada de serviço, com validação do resultado e erro explícito, facilita a troca dos dados de teste por uma API.

6.2 `login.jsx` — chave dinâmica `[name]` (linhas 13 a 25)

```
/**
 * Atualiza o campo digitado e limpa o erro daquele campo.
 * Usa o atributo "name" do input como chave dinâmica no objeto de estado.
 * @param {Event} event - evento de digitação (onChange).
 */
function atualizarCampo(event) {
  const { name, value } = event.target;

  // Copia os campos atuais (spread) e troca só o campo digitado.
  setCampos((valoresAtuais) => ({
    ...valoresAtuais,
    [name]: value, // "login" ou "senha"
  }));

  // Limpa a mensagem de erro do campo que acabou de ser editado.
  setErros((errosAtuais) => ({
    ...errosAtuais,
    [name]: "",
  }));
}
```

Função no sistema: uma única função atende os campos “login” e “senha”: o nome do campo vira a chave, e o valor digitado é guardado nela. Ao digitar, o erro daquele campo some.

Big-O: ler ou escrever `[name]` é **O(1)** em média. O spread copia as k chaves existentes: **O(k)**, com k = 2. Total por tecla digitada: **O(k)**.

Boa prática: nunca alterar o estado diretamente; criar uma cópia nova (imutabilidade) permite ao React detectar a mudança. O custo O(k) é aceitável com k pequeno.

6.3 `CartaoIndicador.jsx` (linha 2)

```
function CartaoIndicador({ perfil, indicador }) {
  // Desestrutura as 4 posições fixas do vetor:
  // [titulo, valor, legenda, nomeDoIcone]
  const [titulo, valor, legenda, icone] = indicador;
```

```
  return (
    <article className="dash-indicador">
      <img src={`/img/dashboard/${perfil}/${icone}.png`} alt="" />
      <h2>{titulo}</h2>
      <strong>{valor}</strong>
      <p>{legenda}</p>
    </article>
  );
}
```

Função no sistema: desenha um cartão de número do topo do dashboard (por exemplo, “Alunos: 126”).

Big-O: ler 4 posições fixas do vetor é **O(1)**.

Boa prática: nomear cada posição na desestruturação torna explícito o significado de cada índice do vetor.

7. Quadro-resumo de Big-O

Arquivo	Estrutura	Operação	Big-O
`dadosAdmin.js`, `dadosProfessor.js`, `dadosResponsavel.js`	Vetores de vetores	Ler por índice	O(1)
`dadosDashboard.js`	Dicionário + vetores	Buscar perfil	O(1) em média
`ConteudoAdmin.jsx`	3 × `.map`	Percorrer listas	O(n)
`ConteudoProfessor.jsx`	2 × `.map`	Percorrer listas	O(n)
`ConteudoResponsavel.jsx`	1 × `.map`	Percorrer lista	O(n)
`MenuLateral.jsx`	`.map`	Percorrer menu	O(n)
`MenuPerfil.jsx`	Vetor + `.map`	Percorrer perfis	O(n)
`Cabecalho.jsx`	Vetor + `for...of`	Percorrer seções	O(n)
`Dashboard.jsx`	Dicionário + `.map`	Buscar componente / percorrer	O(1) / O(n)
`nomes.js`	split → slice → map → join	Gerar iniciais	O(m)
`login.jsx`	`Object.keys`, spread	Validar / atualizar	O(L) / O(k)
`servicoUsuario.js`	2 dicionários	Buscar por perfil	O(1) em média
`usuariosMock.js`	Dicionário	Buscar usuário	O(1) em média
`CartaoIndicador.jsx`	Desestruturação	Ler 4 posições	O(1)